```
# ==========================================
# 1. Import Libraries
# ==========================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ==========================================
# 2. Load Dataset as DataFrame
# ==========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
# 3. Feature-Target Split
# ==========================================
X = df.drop("Price", axis=1)
y = df["Price"]

# ==========================================
# 4. Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ==========================================
# 5. Feature Scaling (VERY IMPORTANT)
# ==========================================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ==========================================
# 6. Build Multilayer Model
# ==========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# ==========================================
# 7. Train Model
# ==========================================
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# ==========================================
# 8. Evaluate Model
# ==========================================
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
```

```
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ==========================================
# 9. Plot Training Curve
# ==========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()

# ==========================================
# 10. Actual vs Predicted Plot
# ==========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

Show hidden output

```
print(history.history.keys())
```

```
dict_keys(['loss', 'mae', 'val_loss', 'val_mae'])
```

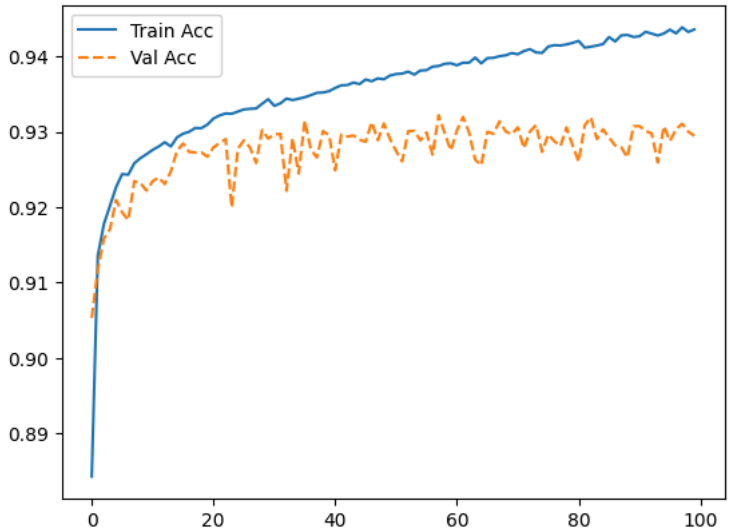
```
# for batch 64
history_64 = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=64,
    verbose=0
)
```

```
# for batch 128
history_128 = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=128,
    verbose=0
)
```

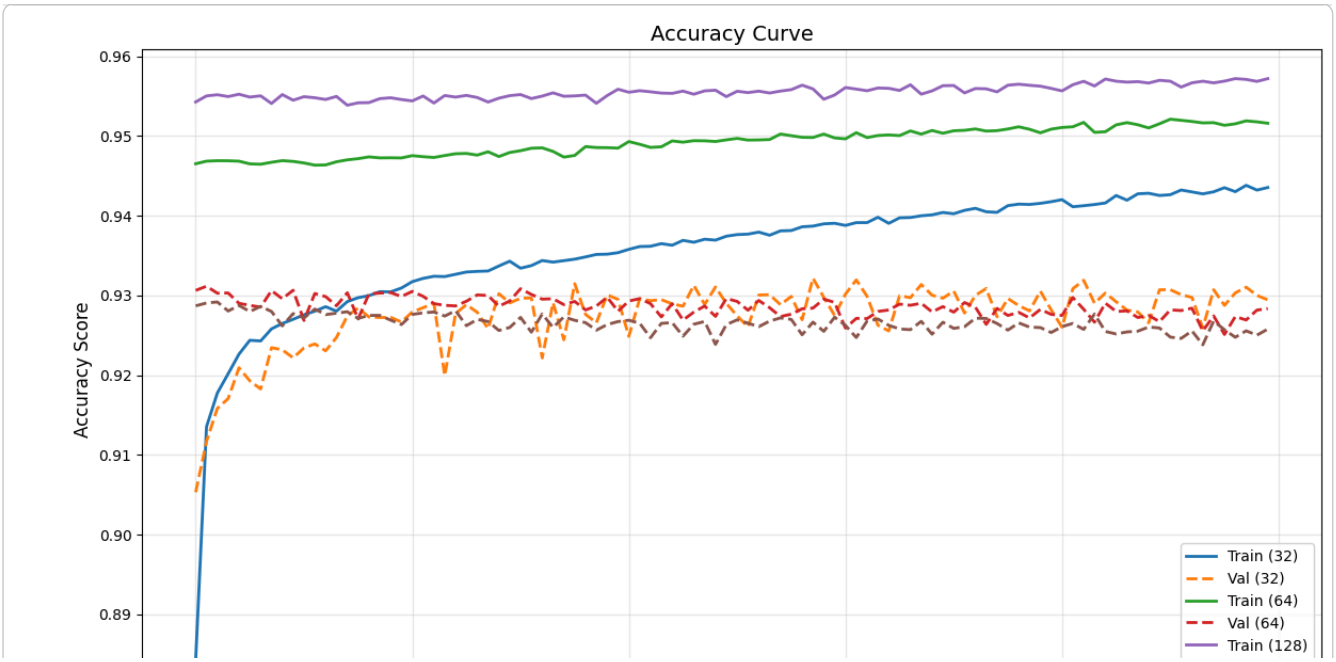
```
print(history.history.keys())
```

```
dict_keys(['loss', 'mae', 'val_loss', 'val_mae'])
```

```
max_val = np.max(y_train)

train_acc_like = 1 - (np.array(history.history['mae']) / max_val)
val_acc_like = 1 - (np.array(history.history['val_mae']) / max_val)

plt.plot(train_acc_like, '-', label='Train Acc')
plt.plot(val_acc_like, '--', label='Val Acc')
plt.legend()
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

max_val = np.max(y_train)

# Convert MAE to accuracy-like
train_32 = 1 - (np.array(history.history['mae']) / max_val)
val_32   = 1 - (np.array(history.history['val_mae']) / max_val)

train_64 = 1 - (np.array(history_64.history['mae']) / max_val)
val_64   = 1 - (np.array(history_64.history['val_mae']) / max_val)

train_128 = 1 - (np.array(history_128.history['mae']) / max_val)
val_128   = 1 - (np.array(history_128.history['val_mae']) / max_val)


plt.figure(figsize=(12,7))

# Batch 32
plt.plot(train_32, '-',  linewidth=2, label='Train (32)')
plt.plot(val_32,   '--', linewidth=2, label='Val (32)')

# Batch 64
plt.plot(train_64, '-',  linewidth=2, label='Train (64)')
plt.plot(val_64,   '--', linewidth=2, label='Val (64)')

# Batch 128
plt.plot(train_128, '-',  linewidth=2, label='Train (128)')
plt.plot(val_128,   '--', linewidth=2, label='Val (128)')


plt.title("Accuracy Curve", fontsize=14)
plt.xlabel("Epochs", fontsize=12)
plt.ylabel("Accuracy Score", fontsize=12)

plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```


Accuracy Curve
Accuracy Score
0.96
0.95
0.94
0.93
0.92
0.91
0.90
0.89
Train (32)
Val (32)
Train (64)
Val (64)
Train (128)